RAMACHANDRA COLLEGE OF ENGINEERING

(Autonomous)

COURSE:OBJECT-ORIENTED PROGRAMMING THROUGH JAVA

Unit-04: Interfaces, Packages and Java Library and Exception Handling

- ✓ **The complete Reference Java, 11th Edition, Herbert Schildt, TMH**

- ✓ **Joy with JAVA, Fundamentals of Object-Oriented Programming, Debasis Samanta, Monalisa Sarma, Cambridge, 2023.**

Interfaces: Introduction, Declaration and Implementation of Interface, Multiple Interfaces, Nested Interfaces, Inheritance of Interfaces, Default Methods in Interfaces, Static Methods in Interface.

Packages and Java Library: Introduction, Defining Package, Importing Packages and Classes into Programs, Path and Class Path, Access Control, Packages in Java SE, Java.lang Package and its Classes, Universal super Class Object Class, Enumeration, class Math, Wrapper Classes, Auto boxing and Auto- un boxing, Java util Classes and Interfaces, Scanner class.

Exception Handling: Introduction, Hierarchy of Standard Exception Classes, Keywords throws and throw, try, catch, and finally Blocks, Multiple Catch Clauses, Class Throwable, Unchecked Exceptions, Checked Exceptions.

LEARNING OBJECTIVE

Make the student able to Understanding to the concepts of interfaces, packages, Java library, and exception handling.

LEARNING OUTCOMES

After attending this session, a student will be able to Apply the concepts of interfaces, packages, Java library, and exception handling

UNIT-IV PART-A :

Interfaces: Introduction, Declaration and Implementation of Interface, Multiple Interfaces, Nested Interfaces, Inheritance of Interfaces, Default Methods in Interfaces, Static Methods in Interface.

PART- A: Interfaces:

Concept-1:Introduction, Declaration and Implementation of Interface

Why use Java interface?

- **There are mainly two reasons to use interface. They are given below.**

1. **It is used to achieve multiple inheritances.**
2. **It is used to achieve abstraction.**

- **Interface in Java having only abstract methods**
- abstract methods means writing only method declaration but not method definition
- **Interface says I will tell you what are the methods you need but I will not implement.**
- **We want to create Concrete class to define methods and concrete class implements interface**
- **Two keywords we are using a.interface b.implements**
- **Syntax:**

```
interface interfacename                    //by using the interface keyword.
{

        void method();                     //declare method

}
```

```
interface Parent // Interface declaration
                 {
    void display(); // Abstract method
}
class Child implements Parent // Class implementing the interface
{


public void display() // Implementing the method from the interface (must be public)
{
        System.out.println("Welcome to RCE");
    }
}
class InterfaceExample
 {
    public static void main(String args[])
{
        Child obj = new Child(); // Creating an object of the Child class
        obj.display(); // Calling the display method
    }
}
```

```
C:\javap>javac InterfaceExample.java

C:\javap>java InterfaceExample
Welcome to RCE
```

PART-A: Interfaces:

Concept-2:Multiple Interfaces & Inheritance of Interfaces

```
interface Parent1           // Define the first interface
{
void display();            //method declare
}
interface Parent2         // Define the second interface
{
 void message();          //method declare
// Implement both interfaces in the Child class
class Child implements Parent1, Parent2
{
public void display() // Implementing the method from the interface (must be public)
{
System.out.println("Welcome to display method");
 }
public void message() // Implementing the method from the interface (must be public)
 {
System.out.println("Welcome to message method");
  }
}
class Sample
 {
    public static void main(String[] args)
 {
Child obj = new Child();    // Create an instance of Child
obj.display();  // Call methods defined in the interfaces
obj.message();
 }
}
```

Abstract class and Methods

- ✓ Abstract class contains atleast one abstract method.i.e., abstract methods and non abstract methods
- ✓ Abstract method only declaration but no definition
- ✓ Implementation of abstract method will be written in derived class
- ✓ Concreate class(contains complete definition or implementation of all the methods)
- ✓ Object can not be created for Abstract class
- ✓ Object can be created for concreate class

Write a JAVA program for abstract class to find areas of different shapes

```
class Shape                 // Abstract base class
{

 void display();                  // Abstract method

}
class Circle extends Shape
                                 // Derived class for Circle
{
double radius=2.25;


void display() // Implementing the abstract method
 {
 double area = 3.14* radius * radius;
 System.out.println("The area of the circle is: " + area);
 }
}
```

Abstract class

concrete class

Write a JAVA program for abstract class to find areas of different shapes

```
class Shape                    // Abstract base class
{

 void display();                    // Abstract method

}
class Circle extends Shape
{                                  // Derived class for Circle
double radius=2.25;



void display() // Implementing the abstract method
 {
 double area = 3.14* radius * radius;
 System.out.println("The area of the circle is: " + area);
 }
}
```

Abstract class

concrete class

```
class Rectangle extends Shape       // Derived class for
```

Rectangle

```
 {
    double length=4.0;
    double breadth=6.0;



    void display()      // Implementing the abstract method
{
        double area = length * breadth;
        System.out.println("The area of the rectangle is: "
+ area);
 }
}
```

concrete class

```
class Main // Main class to test the shapes
{
    public static void main(String[] args)
{
  Circle obj = new Circle(); // Creating object
   Rectangle obj1= new Rectangle(); // Creating object
   obj.display();


     obj1.display();
 }
}
```

```
F:\JAVA LAB_AI&ML\3>javac Shape.java

F:\JAVA LAB_AI&ML\3>java Main
The area of the circle is: 15.89625
The area of the rectangle is: 24.0
```

DIFFERENCE BETWEEN INTERFACE & ABSTRACT

Interface	Abstract class
✓ Interface shouldcontain declaration of methods	✓ Abstract should contain Both declaration and definition of methods
✓ Interface having Interface keyword only for class	✓ Abstract class having Abstract keyword for class and method
✓ Interface is used to achieve multiple inheritance	✓ Abstract class Not Supports multiple inheritance
✓ Interface is used to achieve abstraction	✓ Abstract class is used to achieve abstraction
✓ Interface having only abstract methods	✓ Abstract class having atleast one abstract method and non abstract methods
✓ Interface having impelements keyword	✓ Abstract class having extends keyword
✓ In Interface ,implement keyword is used to import multiples base class properties into single derived class	✓ In Abstract class,extends keyword is used to import single base class properties into single derived class

✓ Syntax: interface
classname
{
abstract method();
}
	✓ Syntax:
	abstract class classname
	{
	abstract method();
	Non-abstract method();
	}

20

PART-A:Interfaces:

Concept-3:Default Methods in Interfaces

- **Before Java 8, interfaces could have only abstract methods. The implementation of these methods has to be provided in a separate class.**
- **So, if a new method is to be added in an interface, then its implementation code has to be provided in the class implementing the same interface.**
- **To overcome this issue, Java 8 has introduced the concept of default methods which allow the interfaces to have methods with implementation without affecting the classes that implement the interface. A default method should not override.**

```
interface Parent
{
Default void display()
{
System.out.println("Welcome to RCE from Parent class");
}
}
class Child implements Parent
{
// can override defaultMethod, but not mandatory
}
class DefaultMethodExample
{
public static void main(String args[])
{
Child obj=new Child(); obj.display();
}
}
```

PART-A:Interfaces:

Concept-4:Static Methods in Interface.

Static Methods in Interface.

- Similar to Default Method in Interface, the static method in an interface can be defined in the interface, but cannot be overridden in Implementation Classes.
- To use a static method, Interface name should be instantiated with it, as it is a part of the Interface only.

```
interface Parent
{
Static void display()
{
System.out.println("Welcome to RCE from Parent class");
}
}
class StaticMethodExample
{
public static void main(String args[])
{
Parent.display();
}
}
```

PART-A:Interfaces:

Concept-5: Nested Interfaces

- **An interface, i.e., declared within another interface or class, is known as a nested interface.**
- **The nested interfaces are used to group related interfaces so that they can be easy to maintain.**
- **The nested interface must be referred to by the outer interface or class. It can't be accessed directly.**

```
class OuterClass
 {
    interface Parent
{
        void display();
    }
}
class innerClass implements OuterClass
 {
         interface child
 {
    public void display()
{
        System.out.println("Welcome to RCE");
    }
class NestedInterfaceExample
 {
    public static void main(String args[ ])
{
        innerclass obj = new innerclass();
        obj.display();
    }
}
```

Unit-IV: PART-B

Exception Handling: Introduction, Hierarchy of Standard Exception Classes, Keywords throws and throw, try, catch, and finally Blocks, Multiple Catch Clauses, Class Throwable, Unchecked Exceptions, Checked Exceptions.

Unit-IV: PART-B:Exception Handling

Concept-1:Introduction, Hierarchy of Standard Exception Classes

- The Exception Handling in Java is one of the powerful mechanism to handle the runtime errors so that the normal flow of the application can be maintained.
- If there is any error occurs in a program automatically rest of the code will not be executed & the program will be terminated abnormally. i.e., Avoiding abnormal termination of program.
- Actually we are use five keywords

i. Try
ii. Catch
iii. Throw
iv. Throws
v. Finally

Exception Handling – How It Works?

When something goes wrong, we don't just stop. We handle it!



An unexpected situation happens in the program (e.g., invalid input, divide by zero, file not found, etc.).

We create an exception object with a meaningful message (e.g., `new InvalidAmountException ("Amount cannot be negative")`)

We throw the exception using the **throw** keyword (e.g., `throw new InvalidAmountException( "Amount cannot be negative")`)

We **catch** the exception and provide a proper response (e.g., show error message, retry, log, etc.).

What is Exception in Java?

Dictionary Meaning: Exception is an abnormal condition.

In Java, an exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

What is Exception Handling?

Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException,

IOException, SQLException, RemoteException, etc.

ADVANTAGE OF EXCEPTION HANDLING

The core advantage of exception handling is to maintain the normal flow of the application. An exception normally disrupts the normal flow of the application; that is why we need to handle exceptions. Let's consider a scenario:

```
statement 1;
statement 2;
statement 3;
statement 4;
statement 5;//exception occurs
statement 6;
statement 7;
statement 8;
statement 9;
statement 10;
```

Suppose there are 10 statements in a Java program and an exception occurs at statement 5; the rest of the code will not be executed, i.e., statements 6 to 10 will not be executed. However, when we perform exception handling, the rest of the statements will be executed. That is why we use exception handling in Java.

Unit-IV: PART-B:Exception Handling

Concept-2:TYPES OF JAVA EXCEPTIONS

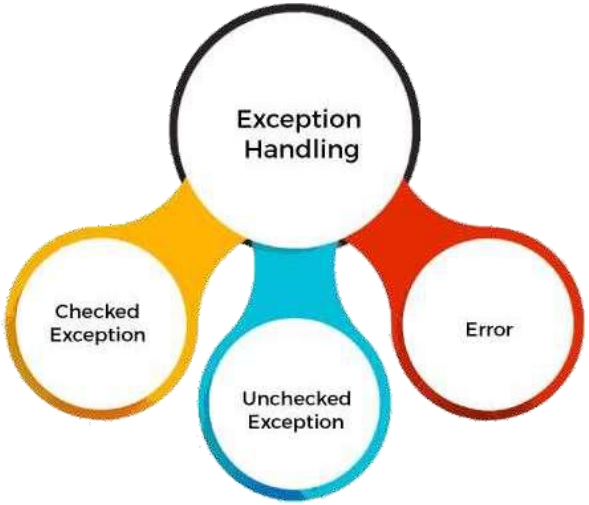
- ✓ **Exception handling is a mechanism in Java that allows developers to manage runtime errors and maintain normal application flow.**
- ✓ **When an error occurs during the execution of a program, an exception object is created, and Java's runtime environment attempts to handle it.**

- ✓ **If the exception is not handled properly, it can lead to program termination.**

TYPES OF JAVA EXCEPTIONS

There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception. However, according to Oracle, there are three types of exceptions namely:

1. CHECKED EXCEPTION
2. UNCHECKED EXCEPTION
3. ERROR



1) Checked Exception

- ✓ The Exceptions which are checked by compiler at the time and It is a compile time Exception
- ✓ They will checked at compile time then whatever the errors are occurred those errors are known as checked exceptions.
- ✓ For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

- ✓ The Exceptions which are not checked by compiler at the time
- ✓ Unchecked exceptions are not checked at compile-time, but they are checked at runtime.
- ✓ It is a Run time Exception
- ✓ For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, NullFormatException etc.

3) Error

- ✓ Error is irrecoverable.
- ✓ Some example of errors are SyntaxError,OutOfMemoryError, VirtualMachineError, AssertionError etc

Unit-IV: PART-B:Exception Handling

Concept-3:COMMON SCENARIOS OF JAVA EXCEPTIONS

COMMON SCENARIOS OF JAVA EXCEPTIONS

There are given some scenarios where unchecked exceptions may occur. They are as follows:

1) where ArithmeticException occurs

If we divide any number by zero , there occurs an ArithmeticException.

```
int a=50/0;
System.out.println(a);//ArithmeticException
```

2) where NullPointerException occurs

We don’t have any initialization but we are accessing the length of the string

```
String name=null;
System.out.println(name.length());//NullPointerException
```

3) where NumberFormatException occurs

Suppose we have a string variable that has characters; it converting this variable into number or digit will cause NumberFormatException.

```
String s="abc";
int i=Integer.parseInt(s);//NumberFormatException
```

4) where ArrayIndexOutOfBoundsException occurs

When an array exceeds to it's size, the ArrayIndexOutOfBoundsException occurs , There may be other reason to occur ArrayIndexOutOf BoundsException

the following statements.

```
int a[]=new int[5];
a[10]=50;
System.out.println(a[10]);//ArrayIndexOutOfBoundsException
```

Unit-IV: PART-B:Exception Handling

Concept-3: Keywords throws and throw, try, catch, and finally Blocks

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

TRY BLOCK

- If an exception occurs at the particular statement in the try block, the rest of the block code will not execute.
- Java try block must be followed by either catch or finally block.
- We have to write code in which we are excepting exception Syntax of try block:

Syntax:

```
try
{
Statements;
}
catch(Exceptionname
{
Statements;
}
}
```

CATCH BLOCK

- What ever the exceptions raised in try block those exceptions will be handle in catch block
- Without try block we should not write the catch block
- Without catch block we should not write try block

The JVM firstly checks whether the exception is handled or not. If exception is not handled, JVM provides a default exception handler that performs the following tasks:

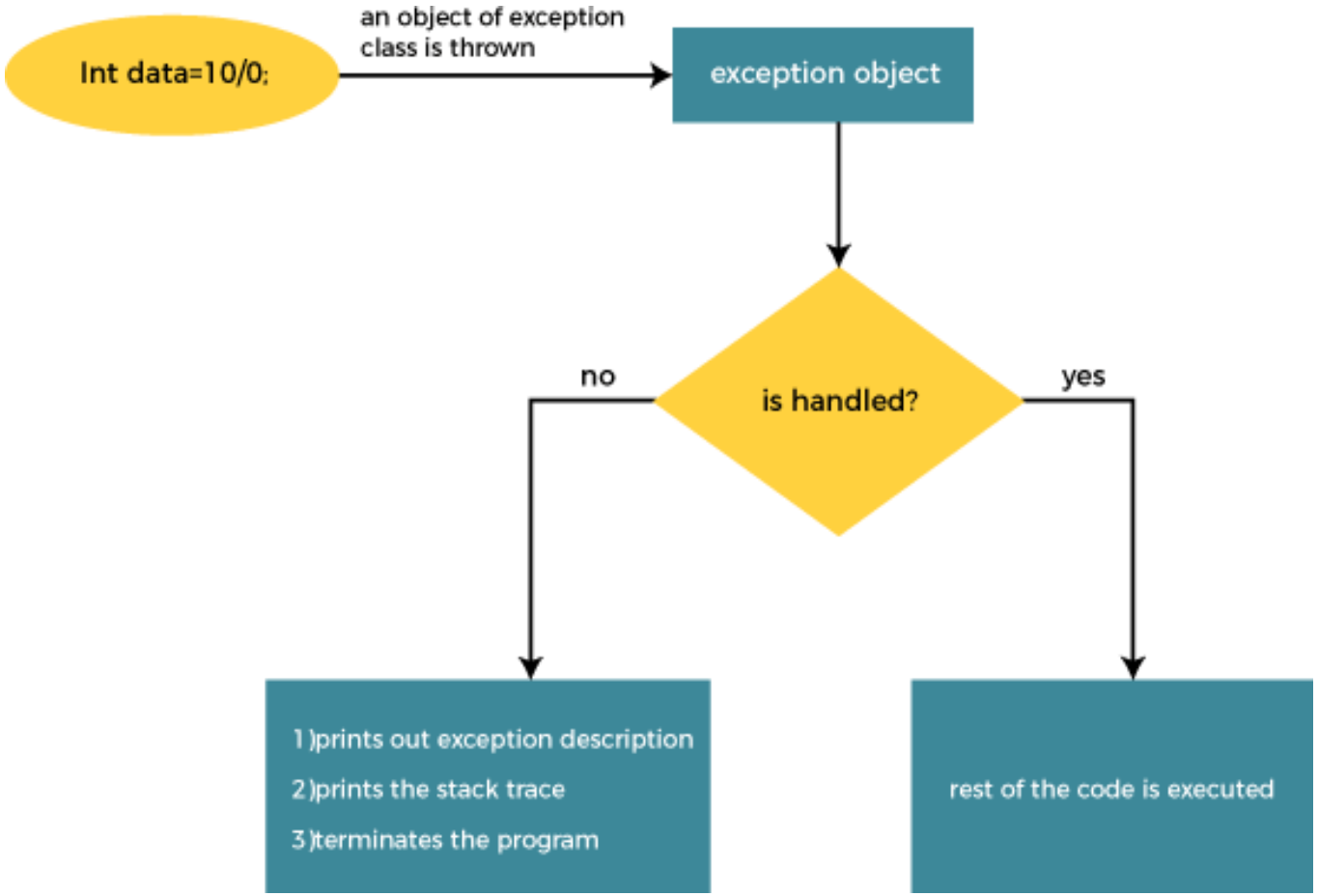


Fig:Internal Working of Java try-catch block

PROBLEM WITHOUT EXCEPTION HANDLING

```
public class WithoutException{
public static void main(String args[])
{
Scanner sc=new Scanner(System.in);
System.out.println("Enter your num number:");
int num=sc.nextInt();
System.out.println("Enter your den number:");
int den=sc.nextInt();
int a=num/den;
System.out.println(a);
}
}
```

```
C:\javap>java WithoutException
Enter your num number:
10
Enter your den number:
0
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at WithoutException.main(WithoutException.java:11)

C:\javap>java WithoutException
Enter your num number:
10
Enter your den number:
5
2
```

PROBLEM WITH EXCEPTION HANDLING

```
import java.util.*;
class WithtrycatchException
{
public static void main(String args[])
{
Scanner sc=new Scanner(System.in);
System.out.println("Enter your num number:");
int num=sc.nextInt();
System.out.println("Enter your den number:");
int den=sc.nextInt();
try
{
int a=num/den;
System.out.println(a);
}
catch(ArithmeticException e)
{
System.out.println("WELCOME TO CATCH");
}
}
}
```

```
C:\javap>javac WithtrycatchException.java

C:\javap>java WithtrycatchException
Enter your num number:
10
Enter your den number:
5
2

C:\javap>java WithtrycatchException
Enter your num number:
25
Enter your den number:
0
WELCOME TO CATCH
```

PROBLEM WITHOUT EXCEPTION HANDLING

```
class WithoutboundException


{
public static void main(String args[])
{
int a[]=new int[10];
a[10]=100;
System.out.println(a);
}
}
}
```

```
C:\javap>javac Withoutbound.java

C:\javap>java Withoutbound
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 10
        at Withoutbound.main(Withoutbound.java:6)
```

PROBLEM WITH EXCEPTION HANDLING

```
class WithboundException
{
public static void main(String args[])
{
try
{
int a[]=new int[10]; a[10]=100;
System.out.println(a);
}
catch(ArrayIndexOutOfBoundsException x)
{
System.out.println("Welcome to ArrayBound Exception");
}
}
}
```

```
C:\javap>javac WithboundException.java

C:\javap>java WithboundException
Welcome to ArrayBound Exception
```

PROBLEM WITHOUT EXCEPTION HANDLING

```
class WithoutNullpointer

{
public static void main(String args[])
{
String name=null;
System.out.println(name.length());
}
}
```

```
C:\javap>javac WithoutNullpointer.java

C:\javap>java WithoutNullpointer
Exception in thread "main" java.lang.NullPointerException
        at WithoutNullpointer.main(WithoutNullpointer.java:6)
```

PROBLEM WITH EXCEPTION HANDLING

```
class WithNullpointerExc

{
public static void main(String args[])
{
try
{
String name=null; System.out.println(name.length());
}
catch(NullPointerException e)
{
System.out.println("Welcome to Nullpointer exception");
}
```

```
C:\javap>javac WithNullpointerExc.java

C:\javap>java WithNullpointerExc
Welcome to Nullpointer exception
```

```
}
```

PROBLEM WITHOUT EXCEPTION HANDLING

```
class WithoutNumberFormat

{
public static void main(String args[])
{
String name="WELCOME TO AIML";
int number=Integer.parseInt(name);
System.out.println(number);
}
}
```

```
C:\javap>javac WithoutNumberFormat.java

C:\javap>java WithoutNumberFormat
Exception in thread "main" java.lang.NumberFormatException: For input string: "WELCOME TO AIML"
        at java.lang.NumberFormatException.forInputString(NumberFormatException.java:65)
        at java.lang.Integer.parseInt(Integer.java:492)
        at java.lang.Integer.parseInt(Integer.java:527)
        at WithoutNumberFormat.main(WithoutNumberFormat.java:6)
```

Problem with Exception Handling

```
class WithNumberFormatExc
{
public static void main(String args[])
{
try
{
String name="WELCOME TO RCE";
int ans=Integer.parseInt(name);
System.out.println(ans);
}
catch(NumberFormatException z)
{
System.out.println("Welcome to Number Format Exception");
}
}
```

```
C:\javap>javac WithNumberFormatExc.java

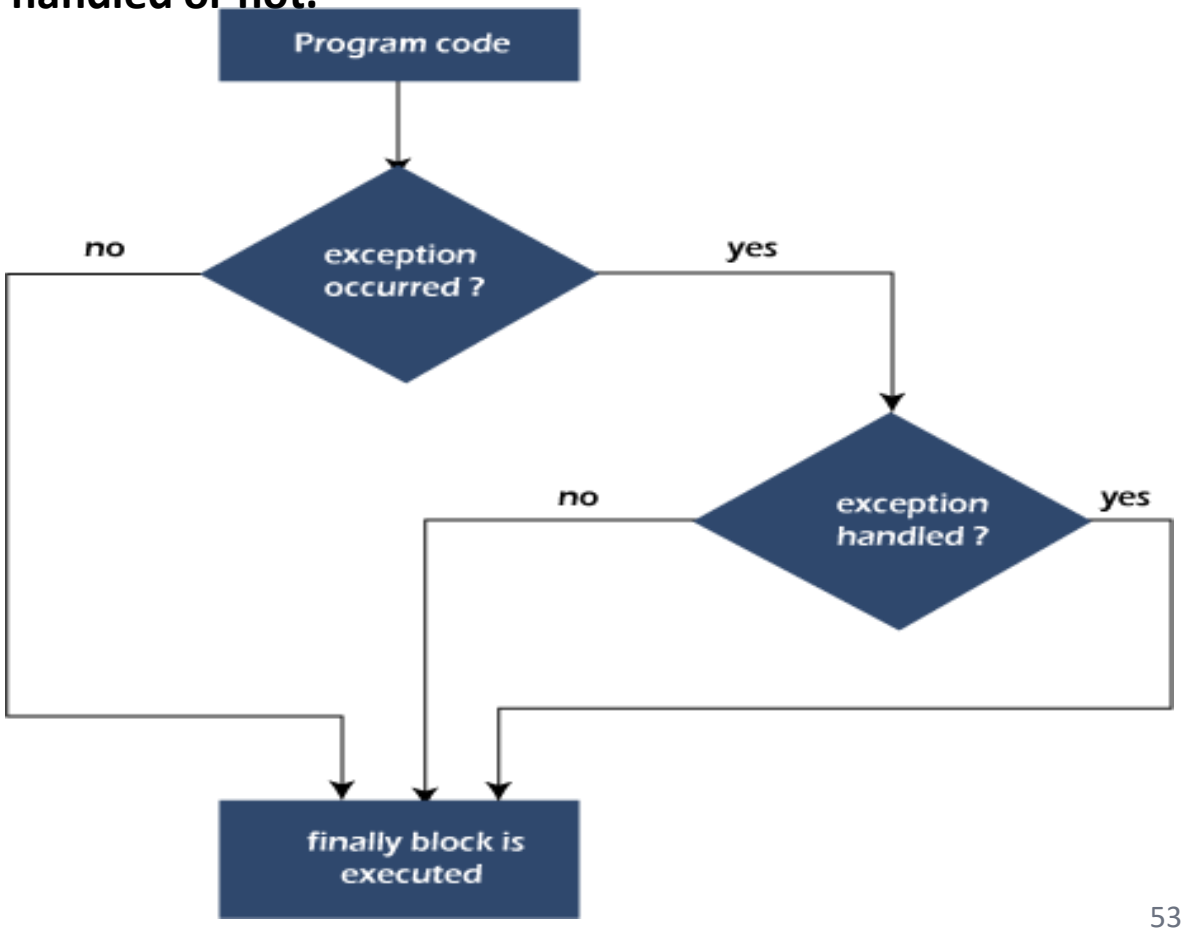
C:\javap>java WithNumberFormatExc
Welcome to Number Format Exception
```

Finally Block

Java finally block is always executed whether an exception is handled or not. Therefore, it contains all the necessary statements that need to be printed regardless of the exception occurs or not.

- **The finally block follows the try-catch block.**
- **Java finally block is always executed whether exception is handled or not.**
- **Java finally block must be followed by try or catch block.**

```
Syntax
try
{
Statements;
}
catch(Exceptionname
{
Statements;
}
finally
{
Statements;
}
```



Case 1: When an exception does not occur

```
class WithfinallyException
{
public static void main(String args[])
{
try
{
int a=100/4; System.out.println(a);
}
catch(ArithmeticException e)
{
System.out.println("Exception Handled by catch block");
}
finally
{
System.out.println("Finally Reached");
}
}
}
```

```
C:\javap>javac WithfinallyException.java

C:\javap>java WithfinallyException
4
Finally Reached
```

Case 2: When an exception occurr but not handled by the catch block

```
class WithoutException
{
public static void main(String args[])
{
try
{
int a=100/0;
System.out.println(a);
}
catch(NullPointerException e)
{
System.out.println("Exception Handled by catch block");
}
finally
{
System.out.println("Finally Reached");
}
}
}
```

```
C:\javap>javac WithoutException.java

C:\javap>java WithoutException
Finally Reached
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at WithoutException.main(WithoutException.java:7)
```

Case 3: When an exception occurs and is handled by the catch block

```
class WithfinallyException
{
public static void main(String args[])
{
try
{
int a=100/0;
System.out.println(a);
}
catch(ArithmeticException e)
{
System.out.println("Exception Handled by catch block");
}
finally
{
System.out.println("Finally Reached");
}
}
}
```

```
C:\javap>javac WithfinallyException.java

C:\javap>java WithfinallyException
Exception Handled by catch block
Finally Reached
```

The throw statement

Use throw to raise an exception at a specific point in your code.

SYNTAX

```
throw new
IllegalArgumentException("Invalid age");
```

Read the statement

`throw` starts the throw operation

`new IllegalArgumentException(...)`

Creates the object; its message adds context.

EXAMPLE: VALIDATE AGE

```
if (age < 18) {
    throw new IllegalArgumentException(
        "Age must be 18 or older");
}
```

The condition is true → Java raises the exception.

WHAT HAPPENS NEXT?

1. **Execution stops here**
   Statements after throw are skipped.
2. **Java looks for catch**
   The nearest matching catch can handle it.
3. **Otherwise, it propagates**
   The caller gets the exception.

Remember: throw raises an exception; throws declares possible exceptions in a method header.

Java Exception Handling – throw

The `throw` keyword is used to explicitly throw an exception from a method or block of code.
It creates an exception object and hands it over to the calling code (which can be caught using try-catch).

How throw Works

1. **Create an exception object**
   (e.g., **new** ArithmeticException("message"))
2. **Use the throw keyword**
   to throw the exception object
3. Control is transferred to the
   nearest **catch** block (if available)

Example

```
class ThrowDemo {
    public static void checkAge(int age) {
        if (age < 18) {
            // Create and throw an exception object
            throw new ArithmeticException("Age must be 18 or above");
        }
        System.out.println("Age is valid: " + age);
    }

    public static void main(String[] args) {
        try {
            checkAge(15);  // This will throw an exception
        } catch (ArithmeticException e) {
            System.out.println("Caught exception: " + e.getMessage());
        }
    }
}
```

Key Points

- **throw** can be used only inside a method or block.
- The object thrown must be an instance of **Throwable** (or its subclass).
- It helps in transferring control to the exception handler (**catch** block).
- It is different from the **throws** keyword. (throws only declares, **throw** actually throws)

In Short

throw = create exception object and hand it over to the caller for handling.

The throws clause

Declare exceptions a method may pass to its caller.

METHOD DECLARATION

```
returnType methodName()
    throws ExceptionType { ... }
```

What does throws tell us?

`throws` lists exception types in the method header

It declares a possibility

It does not throw by itself.

EXAMPLE: A METHOD THAT MAY FAIL

```
static String readFile() throws IOException {
    return Files.readString(
        Path.of("notes.txt"));
}
```

IOException is checked: the caller must catch it or declare it too.

FOLLOW THE EXCEPTION

1. **Method declares it**

   The method may pass this type out.

2. **Exception reaches caller**

   If not caught inside the method.

3. **Caller handles or declares**

   Catch it, or add its own throws.

Remember: throws declares exceptions in a method header; throw raises one in the method body.

Java – throws (Exception Propagation)

The throws keyword is used in a method signature to declare that the method may throw an exception, so the caller knows it needs to handle or further declare it.

What is throws?

- It does not throw the exception itself.
- It declares the type of exception that a method may throw.
- The caller of the method must handle the exception using try-catch or again declare it using **throws**.

Simple Example

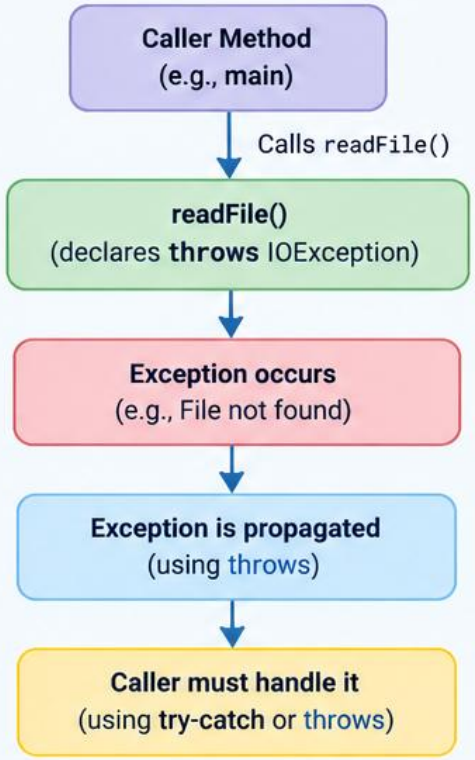
```
import java.io.FileReader;
import java.io.IOException;

public class ReadFileExample {
    public static void readFile() throws IOException {
        FileReader fr = new FileReader("data.txt");
        // may throw IOException
        fr.close();
    }
}
```

What happens here?

The **readFile()** method declares that it may throw **IOException**. It does not catch it. So, the **caller** must handle it.

How it works (Flow Diagram)



Example with try-catch (Handling the exception)

```
public class ReadFileExample {
    public static void main(String[] args) {
        try {
            readFile();  // call the method
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }

    public static void readFile() throws IOException {
        FileReader fr = new FileReader("data.txt");
        fr.close();
    }
}
```

Example with multiple throws

```
public void connectToDatabase() throws SQLException,
                                       ClassNotFoundException {
    Class.forName("com.mysql.cj.jdbc.Driver");  // may throw
    Connection con = DriverManager.getConnection(url, user, pass);
    // may throw SQLException
}
```

When to use throws?

- When the method cannot handle the exception itself.
- When you want to force the caller to handle the exception.
- Useful for reusable methods and library code.

Important Points

- Only checked exceptions can be declared using throws (e.g., IOException, SQLException).
- Runtime exceptions (unchecked) do not need to be declared (e.g., NullPointerException).
- You can declare multiple exceptions separated by commas.

In short

throws = Declare
try-catch = Handle

The throws keyword tells the caller "Hey, this method might throw an exception. Be ready to handle it!"

```
class ThrowsBlockException
{
static void rce() throws ArithmeticException
{
int a=10/0;
System.out.println(a);
}
public static void main(String args[])
{
try
{
rce();
}
catch(ArithmeticException e)
{
System.out.println("Arithmetic Exception handled");
}
}
}
```

```
C:\javap>javac ThrowsBlockException.java

C:\javap>java ThrowsBlockException
Arithmetic Exception handled
```

Unit-IV: PART-B:Exception Handling

Concept-5: Multiple Catch Clauses

,

```
class MultiCatchClauses
{
public static void main(String args[])
{
try
{
int a=100/0;
System.out.println(a);
}
catch(ArithmeticException e)
{
System.out.println("Exception Handled by Airth catch block:"+ e);
}
catch(NullPointerException e)
{
System.out.println("Exception Handled by Null catch block:"+ e);
}
catch(ArrayIndexOutOfBoundsException e)
{
System.out.println("Exception Handled by Outof catch block:"+ e);
}
}
}
```

```
C:\javap>javac MultiCatchClauses.java

C:\javap>java MultiCatchClauses
Exception Handled by Airth catch block:java.lang.ArithmeticException: / by zero
```

UNIT-IV PART-C :

Packages and Java Library : Introduction, Defining Package, Importing Packages and Classes into Programs, Path and Class Path, Access Control, Packages in Java SE, Java.lang Package and its Classes, Universal super Class Object Class, Enumeration, class Math, Wrapper Classes, Auto boxing and Auto-un boxing, Java util Classes and Interfaces, Scanner class.

UNIT-04 PART-C

Concept-01 Packages and Java Library & Advantages

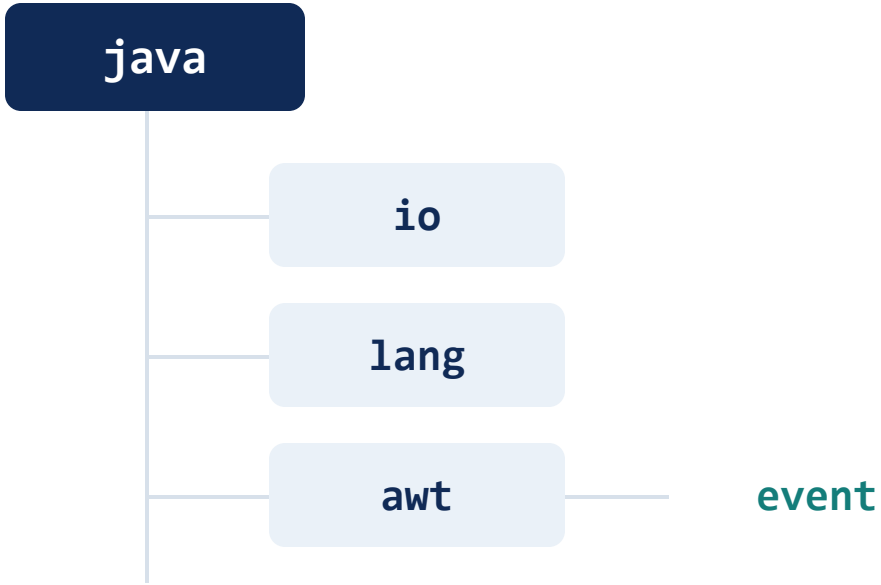
Introduction

- ✓ It is necessary in software development to create several classes and interfaces.
- ✓ After creating these classes and interfaces, it is better if they are divided into some groups depending on their relationship.
- ✓ So these classes and interfaces are stored in some directory.
- ✓ This directory or folder is also known as package.

Package names show a hierarchy

Each dot marks a deeper level in the package structure.

READ THE PACKAGE TREE



DOTS BECOME PATH LEVELS

Package name		Path form
`java.io`	→	`java / io`
`java.lang`	→	`java / lang`
`java.awt.event`	→	`java / awt / event`

A dot separates package levels; it is not part of a folder name.

Example: java.awt.event is nested as java → awt → event.

Advantages of packages

- ✓ **Packages hide the classes and interfaces in a separate sub directory, so accidental deletion of classes and interfaces will not take place**
- ✓ **Two classes in two different packages can have the same name.**
- ✓ **A group of packages is called a library.**
- ✓ **The reusability nature of packages makes programming easy.**

UNIT-04 PART-C

Concept-02 Types of Packages

TYPES OF PACKAGES

There are two types of packages.

I. **Built-in packages.**

II. **User Defined Packages.**

I.Built-in Packages:

These packages consist of a large number of classes. Some of the commonly used built-in packages are:

1) **java.lang:** Contains language support classes (e.g classes which defines primitive data types, math operations). This package is automatically imported.

1) **java.io:** Contains classes for supporting input / output operations.

2) **java.util:** Contains utility classes which implement data structures like Linked List, classes related to date and time

3) **java.net** - Contain classes for supporting networking operations.

4) **applet** – This package provides classes and methods to create and communicate with the applets.

5) **awt** – This package provides classes and methods to create user interfaces.

II.User-defined packages

These are the packages that are defined by the user.

- **A package declaration resides at the top of a java source file. All class are to be placed in a package have common package name.**
- **The name of the package should be followed by the keyword package, declared at the top of the program. Thus, we can define a class belonging to a package.**

```
package com.rce.oops.unit1;
```

Example for User defined package

Declaring a package

Use the package statement to place a class in a named package.

SYNTAX

```
package <package_name>;
```

JAVA SOURCE EXAMPLE

```
package com.rce.oops.unit1;

public class Student {
    // class code goes here
}
```

HOW TO READ IT

```
package com.rce.oops.unit1;
```

Names the package that contains Student.

Where it goes

Put the package declaration before imports and class declarations. Comments may come first.

Naming style

Package names are conventionally written in lowercase.

Typical source path

```
src / com /rce /oops /unit1 / Student.java
```

The folder path mirrors the package name.

UNIT-04 PART-C

Concept-03 Importing Packages and Classes into Programs

Importing Packages and Classes into Programs

How to access package from another package?

There are three ways to access the package from outside the package.

1) **import package.*;**
2) **import package.classname;**
3) **fully qualified name.**

Using a class from another package

Choose an import style—or write the class's full package name.

EXAMPLE CLASS

Assume Student is public and declared in package school.people.

1 Import all types

```
import com.rce.oops.*;
Student s = new Student();
```

2 Import one class

```
import com.rce.oops.Student;
Student s = new Student();
```

3 Use the full name

```
com.rce.oops.Student s =
 com.rce.oops.Student.Student();
```

Access note Student must be public. The * import covers this package only; it does not include subpackages.

UNIT-03 PART-C

Concept-04:Access Control & Packages in Java SE

Who can access a member?

Access depends on the modifier and where the accessing code is located.

VISIBILITY MATRIX

Access location	private	package-private (no keyword)	protected	public
Inside declaring class	YES	YES	YES	YES
Other class, same package	NO	YES	YES	YES
Subclass, different package	NO	NO	YES	YES
Unrelated class, different package	NO	NO	NO	YES

QUICK EXAMPLES

```
private int pin;
```

Only code inside the declaring class can access pin.

```
int score;
```

No modifier means package-private: same-package code can access score.

```
protected void show() { }
```

A subclass in another package can use an inherited protected member.

Remember

Cross-package protected access follows a subclass rule. Public access also requires the declaring class to be accessible.

Packages in Java SE

Java Standard Edition provides 14 packages namely

- ✓ **applet – This package provides classes and methods to create and communicate with the applets.**
- ✓ **awt – This package provides classes and methods to create userinterfaces.**
- ✓ **io – This package contains classes and methods to read and write data standard input and output devices, streams and files.**
- ✓ **lang – This package contains the fundamental classes, methods and interfaces of Java language.**
- ✓ **math – This package contains classes and methods which helps you to perform arithmetic operations using the Java language.**
- ✓ **net – This package provides classes to implement networking applications.**
- ✓ **rmi – This package provides classes, methods, and interfaces for Remote Method Invocation.**

- **security** – **This package provides classes and interfaces for security framework**

- **sql – This package provides classes and methods to access and process the data stored in a data source.**

- **text – This package provides classes and interfaces to handle text.**

- **util – This package contains collection framework, collection classes, classes related to date and time, event model, internationalization, and miscellaneous utility classes.**

UNIT-04 PART-C

Concept-05 HOW TO CREATE SET PATH & CLASS PATH

How to Create SET PATH & CLASS PATH

PATH and CLASSPATH are environment variables used by the Java compiler and runtime to find the required files (JDK tools, classes, and libraries).

1 Setting PATH

PATH tells the operating system where to find Java tools like javac, java, etc.

Steps	Windows (Command Prompt)
1 Find the JDK installation path (where bin folder exists).	Example: `C:\Program Files\Java\jdk-17\bin`
2 Open System Properties → Advanced → Environment Variables → System variables.	Add the JDK bin path to PATH. `Path = %Path%;C:\Program Files\Java\jdk-17\bin`

3 Click OK and close all windows. Open a new command prompt and verify.	Run these commands:
	`java -version`
	`javac -version`

Linux / macOS (Terminal)

1 Edit your shell profile file (e.g., ~/.bashrc or ~/.zshrc).	Add the following line (replace with your JDK path): `export PATH=$PATH:/usr/lib/jvm/jdk-17/bin`
2 Apply the changes.	Run: `source ~/.bashrc  # or source ~/.zshrc`
3 Verify.	`java -version` `javac -version`

2 Setting CLASSPATH

CLASSPATH tells Java where to find your compiled classes and external libraries (JAR files).

Steps	Windows (Command Prompt)
1 Create a folder for your project (e.g., C:\JavaProjects\Demo).	Example: `C:\JavaProjects\Demo`
2 Set CLASSPATH to include the current directory (.) and any required JAR files.	Add the following to System variables: `CLASSPATH = %JAVA_HOME%\lib\dt.jar;%JAVA_HOME%\lib\tools.jar;.;`
3 Save and close. Open a new command prompt and verify.	Run: `echo %CLASSPATH%`

Linux / macOS (Terminal)

1 Set CLASSPATH in your shell profile (e.g., ~/.bashrc or ~/.zshrc).	Add the following line (replace with your JAR path if needed): `export CLASSPATH=.:/path/to/your/lib/*`
2 Apply the changes.	Run: `source ~/.bashrc  # or source ~/.zshrc`
3 Verify.	`echo $CLASSPATH`

Key Points to Remember

- PATH is used to find Java executables (java, javac, etc.).
- CLASSPATH is used to find compiled classes and JAR files.
- Use ";" as separator in Windows and ":" as separator in Linux/macOS.

Example Final Setup (Windows)

```
PATH = %Path%;C:\Program Files\Java\jdk-17\bin
CLASSPATH = %JAVA_HOME%\lib\dt.jar;%JAVA_HOME%\lib\tools.jar;.;
```

UNIT-04 PART-C

Concept-06 Wrapper Classes and Auto-boxing and Auto-unboxing

- **The wrapper class in Java provides the mechanism to convert primitive into object and object into primitive.**
- **The wrapper classes in Java are used to convert primitive types (int, char, float, etc) into corresponding objects.**
- **Autoboxing and Unboxing are the features included in Java 1.5**
- **Autoboxing and Unboxing feature convert primitives into objects and objects into primitives automatically.**
- **The automatic conversion of primitive into object is known as Autoboxing and vice-versa Unboxing.**

Auto Boxing or Boxing:

1. **Autoboxing is the process of converting a primitive type data into its corresponding wrapper class object instance**

2. **Unboxing:**

3. **Unboxing is the process of converting a Objective instance into a primitive type**

Java performs these conversions automatically, which is called autoboxing and auto-unboxing

Primitive types and wrapper classes

Each Java primitive has a corresponding wrapper class.

Primitive type	Size	Wrapper class	Example
boolean	Not fixed by Java	**Boolean**	`boolean ready = true;`
char	16 bits	**Character**	`char grade = 'A';`
byte	8 bits	**Byte**	`byte age = 18;`
short	16 bits	**Short**	`short year = 2026;`
int	32 bits	**Integer**	`int count = 42;`
long	64 bits	**Long**	`long distance = 9_000_000_000L;`
float	32 bits	**Float**	`float price = 19.99F;`
double	64 bits	**Double**	`double rate = 3.14159;`

Autoboxing primitive → wrapper **Unboxing** wrapper → primitive

Wrappers are useful where Java APIs require objects, such as ArrayList<Integer>.

```java
//AutoBoxing Example
class BoxingExample
{
 public static void main(String args[])
 {
     int a=50;
     Integer obj=new Integer(a);//Boxing
     System.out.println("Boxing: "+obj);


 }
```

```
C:\javap>javac BoxingExample.java

C:\javap>java BoxingExample
Boxing: 50
```

```java
//UnBoxing Example
class UnboxingExample
{
public static void main(String args[])
{
     Integer obj=new Integer(50);
     int a=obj;
     System.out.println("Unboxing :" +a);
}
}
```

```
C:\javap>javac UnboxingExample.java

C:\javap>java UnboxingExample
Unboxing :50
```

UNIT-04 PART-C

Concept-07 Math Class

- **Java Math Class in Java contains certain methods to perform different numeric operations, is exponential, square root, logarithmic, trigonometric functions.**
- **Java Math class provides several methods to work on math calculations like sqrt(),min(), max(),avg(), sin(), cos(), tan(), round(),floor(), abs() etc.**
- **Arithmetic operations like increment, decrement, divide, absolute value, It should be checked against the maximum and minimum value as appropriate.**

Common methods in java.lang.Math

Call each static method as Math.method(...).

Method	What it does	Example → result
Math.abs(x)	Returns the absolute value	`Math.abs(-7) → 7`
Math.max(a, b)	Returns the larger value	`Math.max(12, 20) → 20`
Math.min(a, b)	Returns the smaller value	`Math.min(12, 20) → 12`
Math.round(x)	Rounds to the nearest integer	`Math.round(4.6) → 5L`
Math.sqrt(x)	Returns the square root	`Math.sqrt(81) → 9.0`
Math.cbrt(x)	Returns the cube root	`Math.cbrt(27) → 3.0`
Math.pow(a, b)	Raises a to the power b	`Math.pow(2, 3) → 8.0`
Math.sin(x)	Returns the sine of x	`Math.sin(Math.toRadians(30)) ≈ 0.5`
Math.cos(x)	Returns the cosine of x	`Math.cos(Math.toRadians(60)) ≈ 0.5`
Math.tan(x)	Returns the tangent of x	`Math.tan(Math.toRadians(45)) ≈ 1.0`

Radians sin, cos, and tan expect radians. Use Math.toRadians(degrees) to convert degrees.

```
class MathClassExample

 {
public static void main(String args[])
{
 double x = 28;
 double y = 4;
 System.out.println("Maximum number of x and y is: " + Math.max(x, y));
 System.out.println("Square root of y is: " + Math.sqrt(y));
 System.out.println("Power of x and y is: " + Math.pow(x, y));
 System.out.println("Logarithm of x is: " + Math.log(x));
 System.out.println("Logarithm of y is: " + Math.log(y));
 System.out.println("Cube root of x is: " + Math.cbrt(x));
 System.out.println("Round of y is: " + Math.round(11.78));
 System.out.println("Sin of x is: " + Math.sin(90));
 System.out.println("Cos of x is: " + Math.cos(0));
 }
 }
```

```
C:\javap>javac MathClassExample.java

C:\javap>java MathClassExample
Maximum number of x and y is: 28.0
Square root of y is: 2.0
Power of x and y is: 614656.0
Logarithm of x is: 3.332204510175204
Logarithm of y is: 1.3862943611198906
Cube root of x is: 3.036588971875662 7
Round of y is: 12
Sin of x is: 0.8939966636005579
Cos of x is: 1.0
```

UNIT-04 PART-C

Concept-08 Enumeration in java

Enumeration in java

- The Enum in Java is a data type which contains a fixed set of constants.
- It can be used for days of the week, directions, season, colors etc.
- enum constants in capital letters.

- The enum can be defined within or outside the class because it is similar to a class.
- The semicolon (;) at the end of the enum constants are optional.
- For example:

```
enum Colors { RED,GREEN,BLUE,BLACK,YELLOW,PURPLE }
enum Day {
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY,
    FRIDAY, SATURDAY, SUNDAY
}
```

Few More Examples

- **Day:** MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY
- **Month:** JANUARY, FEBRUARY, MARCH, APRIL, MAY, JUNE, JULY, AUGUST, SEPTEMBER, OCTOBER, NOVEMBER, DECEMBER
- **Direction:** NORTH, SOUTH, EAST, WEST
- **TrafficLight:** RED, YELLOW, GREEN
- **Season:** SPRING, SUMMER, AUTUMN, WINTER
- **OrderStatus:** NEW, CONFIRMED, PROCESSING, SHIPPED, DELIVERED, CANCELLED
- **PaymentStatus:** PENDING, COMPLETED, FAILED, REFUNDED
- **TicketStatus:** OPEN, IN_PROGRESS, RESOLVED, CLOSED
- **Priority:** LOW, MEDIUM, HIGH, CRITICAL
- **UserRole:** ADMIN, MANAGER, TEACHER, STUDENT, GUEST
- **FileType:** TXT, PDF, DOCX, JPG, PNG
- **Grade:** A, B, C, D, F
- **Size:** SMALL, MEDIUM, LARGE, EXTRA_LARGE
- **Operation:** ADD, SUBTRACT, MULTIPLY, DIVIDE
- **CardSuit:** HEARTS, DIAMONDS, CLUBS, SPADES
- **Planet:** MERCURY, VENUS, EARTH, MARS, JUPITER, SATURN, URANUS, NEPTUNE
- **ConnectionState:** CONNECTED, DISCONNECTED, CONNECTING, FAILED
- **HttpMethod:** GET, POST, PUT, PATCH, DELETE

Java Enum Example:

```
enum Days{ Sunday,Monday,Tuesday,Wdnesday,Thursday,Friday,Saturday}
class EnumExample
{
   public static void main(String args[])
{
        Days a[] = Days.values(); // get all enum constants


        for (int i = 0; i <a.length; i++)
{
            System.out.println(a[i]);
        }
    }
}
```

```
C:\javap>javac EnumExample.java

C:\javap>java EnumExample
Sunday
Monday
Tuesday
Wdnesday
Thursday
Friday
Saturday
```

Java Enum Example:

```
class EnumDemo1
 {
       enum Colors{ RED, BLUE, BLACK, GREEN }


    public static void main(String args[])
 {
        Colors a[] = Colors.values(); // get all enum constants


        for (int i = 0; i <a.length; i++)
{
            System.out.println(a[i])
        }
    }


}
```

```
C:\javap>javac EnumDemo1.java

C:\javap>java EnumDemo1
RED
BLUE
BLACK
GREEN
```

Java Enum Example: Defined inside class

```
enum Days{ Sunday,Monday,Tuesday,Wdnesday,Thursday,Friday,Saturday;}


class EnumSample
 {
    public static void main(String args[])
 {
        Days a= Days.Thursday; // get all enum constants
       System.out.println(a);
        }
    }
```

```
C:\javap>javac EnumSample.java

C:\javap>java EnumSample
Thursday
```

Java Enum Example: main method inside Enum

```
enum Colors{
        RED,BLUE,BLACK,GREEN;
        }
        public static void main(String[] args)
        {
                Colors c=Colors.RED;
                System.out.println(c);
        }
}
```

Output:

RED

UNIT-04 PART-C

Concept-09 Java. lang Package and its Classes

java.lang Package – Important Classes

The **java.lang** package is automatically imported in every Java program.
It provides many essential classes that are commonly used in Java applications.

Class	Purpose	Class	Purpose
Object	Root class of all Java classes. Provides common methods like toString(), equals(), hashCode(), etc.	**Boolean**	Wrapper class for boolean.
String	Represents a sequence of characters. Provides many methods for string manipulation.	**Long**	Wrapper class for long.
StringBuffer	Mutable sequence of characters (thread-safe).	**Float**	Wrapper class for float.
StringBuilder	Mutable sequence of characters (faster than StringBuffer).	**Byte**	Wrapper class for byte.
Math	Provides mathematical operations like abs(), max(), min(), sqrt(), pow(), etc.	**Short**	Wrapper class for short.
System	Provides input, output and system-related operations. (e.g., System.out, System.in, System.exit())	**Number**	Superclass of numeric wrapper classes.
Integer	Wrapper class for int.	**Thread**	Used for multithreading and concurrent programming.
Double	Wrapper class for double.	**Exception**	Base class for many exceptions.
Character	Wrapper class for char.	**RuntimeException**	Base class for unchecked exceptions.
		Throwable	Parent class of Error and Exception.

Key Point: | java.lang → automatically available → no explicit import required.

You can directly use classes like:
`String`, `Math`, `System`, `Integer`, `Character`, `Thread`, `Object`, `Exception`, etc.

java.lang Package – Important Interfaces

The **java.lang** package contains several important interfaces that are commonly used in Java programs.

These interfaces define a contract (set of methods) that implementing classes must follow.

Interface	Purpose	Key Methods (Examples)
Comparable<T>	Used to compare objects for ordering.	`int compareTo(T o)` (returns -1, 0, 1)
Comparator<T>	Defines custom comparison logic.	`int compare(T o1, T o2)` (returns -1, 0, 1)
Iterable<T>	Allows iteration over a collection.	`Iterator<T> iterator()`
Iterator<T>	Traverses elements one by one.	`boolean hasNext()` `T next()`
Cloneable	Marks a class as cloneable.	`Object clone()` (from Object class)
Serializable	Marks a class as serializable (for file/network).	(No methods – marker interface)

Interface	Purpose	Key Methods (Examples)
CharSequence	Represents a sequence of characters.	`int length()` `char charAt(int index)` `CharSequence subSequence(int, int)`
AutoCloseable	Used in try-with-resources for automatic resource management.	`void close()` (from AutoCloseable)
Functional Interface (Runnable)	Represents a task with no arguments and no return value.	`void run()`
Functional Interface (Callable<V>)	Represents a task that returns a value and can throw an exception.	`V call() throws Exception`
Functional Interface (Comparator)	(Already covered above, part of java.util, but in java.lang hierarchy conceptually related)	`int compare(T o1, T o2)`
Functional Interface (Supplier<T>)	Provides a result without any input.	`T get()`

Key Points to Remember

- **Interfaces define a contract**; classes implement them.
- Many of these interfaces are used in collections, multithreading and functional programming.
- Functional interfaces (like Runnable, Callable, Supplier) are used in lambda expressions (Java 8+).

Example Use Cases

- Comparable → Natural ordering (e.g., Student, Employee)
- Iterator → Loop through a collection
- Runnable → Create a thread
- AutoCloseable → try-with-resources for files, sockets, etc.

UNIT-04 PART-C

Concept-10:Java util Classes and Interfaces

Classes of java.util package

The **java.util** package provides many useful utility classes for working with collections, dates, concurrency, and other common tasks.

Class	Purpose
Arrays	Provides utility methods for working with arrays.
ArrayList	Resizable array implementation of List.
ArrayDeque	Double-ended queue implementation.
BitSet	Efficient way to store and manipulate bits.
Calendar	Provides methods to work with dates and times.
Collections	Utility methods for collections (e.g., sort, max, min).
Currency	Represents a currency (monetary value).
Date	Represents date and time (legacy class).
Dictionary	Legacy class for key-value pairs (Hashtable).

Class	Purpose
EnumMap	Map with enum keys.
EnumSet	Set implementation for enum values.
Formatter	Formats text, numbers, and dates.
HashMap	Hash table implementation of Map.
HashSet	Hash table implementation of Set.
Hashtable	Synchronized map (legacy class).
IdentityHashMap	Map using reference equality.
LinkedList	Doubly linked list implementation.
Locale	Represents a geographical, political, or cultural region.
PriorityQueue	Priority queue implementation.
Random	Generates random numbers.
Scanner	Scans input from various sources.
StringTokenizer	Splits a string into tokens.
Vector	Resizable array (legacy class).
TreeMap	Red-black tree based map.
TreeSet	Red-black tree based set.
Timer	Schedules tasks to be executed later.

Remember: These classes are part of the **java.util** package and help in handling collections, dates, and other utility operations in Java programs.

java.util Package – Important Interfaces

The **java.util** package provides many useful interfaces to work with collections, dates, and other utility operations.

Here are some important interfaces commonly used in Java programs.

Interface	Purpose	Key Methods (Examples)
Collection<E>	Root interface for all collection classes.	`add()`, `remove()`, `size()`, `iterator()`
List<E>	Represents an ordered collection (allows duplicates).	`add(index, e)`, `get(index)`, `remove(index)`, `size()`
Set<E>	Represents a collection with no duplicate elements.	`add()`, `remove()`, `contains()`, `size()`
Queue<E>	Represents a collection for holding elements in FIFO order.	`offer()`, `poll()`, `peek()`, `size()`
Deque<E>	Represents a double-ended queue (both ends).	`addFirst()`, `addLast()`, `removeFirst()`, `removeLast()`
Map<K,V>	Represents a mapping of key-value pairs.	`put()`, `get()`, `remove()`, `containsKey()`, `keySet()`
SortedSet<E>	A Set with elements in ascending order.	`first()`, `last()`, `headSet()`, `tailSet()`
SortedMap<K,V>	A Map with keys in ascending order.	`firstKey()`, `lastKey()`, `headMap()`, `tailMap()`

Interface	Purpose	Key Methods (Examples)
Iterator<E>	Traverses elements of a collection.	`hasNext()`, `next()`, `remove()`
ListIterator<E>	Iterator for List (supports bidirectional traversal).	`hasNext()`, `hasPrevious()`, `next()`, `previous()`
Enumeration<E>	Legacy interface for traversing elements.	`hasMoreElements()`, `nextElement()`
Comparator<T>	Defines custom sorting logic (comparison).	`compare(T o1, T o2)`
Comparable<T>	Natural ordering of objects (implements compareTo()).	`compareTo(T o)`
Spliterator<T>	Supports parallel traversal of elements.	`tryAdvance()`, `forEachRemaining()`
Iterable<T>	Allows an object to be used in enhanced for loop.	`iterator()`
RandomAccess	Marker interface for random-access lists (e.g., ArrayList).	(No methods – marker interface)

Key Points to Remember

- These interfaces help in working with collections, sorting, iteration, and mapping.
- List, Set, Queue, Deque, Map are the most frequently used collection interfaces.
- Comparator and Comparable are used for custom and natural sorting.
- Iterable, Iterator and Spliterator help in traversing elements.

Example Use Cases

- List → Store a list of student names.
- Set → Store unique email IDs.
- Map → Store key-value pairs (e.g., roll number → name).
- Comparator → Sort students by marks.
- Iterator → Iterate through a collection.

Thank You..!